

Gradient Boosting

Gradient Boosting is a powerful ensemble machine learning technique that builds a predictive model by combining multiple weak learners, typically decision trees, in a sequential manner. Each new tree introduced into the model aims to correct errors made by the previous trees. This method lends itself well to various predictive tasks, including classification and regression. Here's a detailed breakdown of the gradient boosting algorithm and its training process:

Detailed Algorithm Steps

1. Initialization:

- Let the initial model $F_0(x)$ be a constant value equal to the average of the target values (for regression) or the log-odds (for classification):

$$F_0 = \arg \min_c \sum_{i=1}^n L(y_i, c).$$

2. Iterative Process:

- For $m = 1$ to M (number of boosting rounds):

1. Compute Pseudo-Residuals:

- For each instance i , compute the pseudo-residuals, which are typically the negative gradients of the loss function L with respect to the predictions:

$$r_i^m = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F^{m-1}}$$

– Example (for a regression task):

Using the L2 (squared error) loss function, the goal is to minimize the mean squared error between the predicted values and the actual target values. The L2 loss function is given by:

$$L(y, F(x)) = \frac{1}{2} (y - F(x))^2$$

The gradient of this loss function with respect to the model's predictions $F(x)$ is:

$$\frac{\partial L(y, F(x))}{\partial F(x)} = F(x) - y$$

Therefore, the pseudo-residuals r_i^m for each instance i at the m^{th} boosting iteration are the negative gradients of the loss function with respect to the current predictions:

$$r_i^m = - \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = y_i - F^{m-1}(x_i)$$

– Example (for a binary classification task):

We often use the logistic (log-loss) function. Let y be the binary label (0 or 1), and $p = P(y=1|x)$ be the predicted probability that $y = 1$. The log-loss function is given by:

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

For gradient boosting in binary classification, we need to work with the predicted logits rather than the probabilities. The logit function $F(x)$ (also called the log-odds) is related to the predicted probability by the logistic function:

$$p = \frac{1}{1+e^{-F(x)}}$$

The gradient of the log-loss function with respect to the logit $F(x)$ is:

$$\frac{\partial L(y, p)}{\partial F(x)} = p - y$$

Therefore, the pseudo-residuals r_i^m for each instance i at the m^{th} boosting iteration are:

$$r_i^m = -\frac{\partial L(y_i, p_i)}{\partial F(x_i)} = y_i - p_i$$

where y_i is the actual binary label (0 or 1), p_i is the predicted probability after $m-1$ iterations.

2. Fit Weak Learner:

- Fit a weak learner (e.g., a shallow decision tree) to the pseudo-residuals. This gives a new model $h_m(x)$ that approximates the residuals.
- Why focusing on residuals is essential?
Residuals represent the errors made by the current model. By training a new weak learner (typically a decision tree) on these residuals, the model focuses specifically on the parts of the data where the current predictions are wrong.

Gradient boosting can be interpreted as a form of gradient descent optimization. The residuals can be seen as the negative gradients of the loss function with respect to the model's predictions. By training on these residuals, we're essentially performing a gradient step in the function space, moving our ensemble model in the direction that reduces the loss the most.

3. Update Model:

- Update the current model by adding the contribution from the new weak learner, scaled by a learning rate ν :

$$F^m(x) = F^{m-1}(x) + \nu \cdot h_m(x)$$

3. Final Model

- The final model after M iterations is $F_M(x) = F_0(x) + \sum_{m=1}^M \nu \cdot h_m(x)$.

Key Hyperparameters

- **Number of Trees (M):** The total number of iterations or weak learners to be added.
- **Learning Rate (v):** A shrinkage parameter that scales the contribution of each tree. It controls the step size at each iteration and helps prevent overfitting.
- **Tree Depth:** The maximum depth of each decision tree. It controls the complexity of each weak learner and, by extension, the overall complexity of the ensemble model.
- **Min Samples Split:** The minimum number of samples required to split an internal node in the decision tree.
- **Min Samples Leaf:** The minimum number of samples required to be at a leaf node.

Practical Considerations

- **Regularization:** To prevent overfitting, it is common to use techniques such as subsampling (training each tree on a random subset of the data) and parameter tuning.
- **Early Stopping:** To determine the optimal number of trees, it is beneficial to monitor the model's performance on a validation set and use early stopping if the performance starts to degrade.